

[Pointers]

1. Write a C program to check whether a pair of numbers is amicable or not, using pointers.

Example: 220 & 284 are amicable numbers then the factors of 220: 1, 2, 4, 5, 10, 11, 20, 22, 44, 55, 110 then $1 + 2 + 4 + 5 + 10 + 11 + 20 + 22 + 44 + 55 + 110 = 284$ and Now, 284: 1, 2, 4, 71, 142 $1 + 2 + 4 + 71 + 142 = 220$.

2. Write a C program to perform basic arithmetic operations (addition, subtraction, multiplication, division) using an array of function pointers.
3. Write a C program to calculate the volume of the following shapes:
 - Cube
 - Cuboid
 - Sphere
 - Cylinder
 - Cone.

Let the user select a shape, input dimensions in main(), and call shape-specific functions (using appropriate parameters) to compute the volume by using pointers.

4. Write a program in C to print all permutations of a given string using pointers.
Expected Output :

The permutations of the string are :

abcd abdc acbd acdb adcb adbc bacd badc bcad bcda bdca bdac cbad cbda
cabd cadb cdab cdba dbca dbac dcba dcab dacb dabc

5. Write a C program to input n numbers using dynamic memory allocation and find the largest element using a pointer.
6. Write a C program to calculate the length of a string using a pointer?
7. Write a C program to count and print the number of vowels and consonants in a string using a pointer?
8. Write a C program to print all the alphabets (both upper and lower case) using a pointer?
9. Write a C program to accept a string and check whether it is a palindrome or not using pointers.

10. Write a C program to dynamically allocate a 2D array using an array of pointers.

11. Write a C program using pointer to strings that accepts the name of an animal and a bird and returns the names in plural.

12. Write a C program to demonstrate how to perform arithmetic on void pointers (if supported by compiler).

13. Write a C program to read multiple lines of text until EOF, storing each line using malloc, and then print them in reverse order.

- Use getline() or fgets() to read each line.
- Use malloc() to save each line, and realloc() to manage the array of line pointers.

14. Write a C program to implement a stack using pointers with the following operations:

- push
- pop
- display

15. Write a C program to allocate and free a 2D matrix using double pointers (int **matrix).

16. int main()

{

int arr[6]={11,22,33,44,55,66};

int *ptr=arr;

printf("%d\n",*ptr++);

printf("%d\n",++*ptr);

printf("%d\n",*++ptr);

ptr[1]++;

ptr++;

ptr[-1]++;

```
ptr[0]++;  
printf(“%d %d %d %d %d %d\n”,arr[0],arr[1],arr[2],arr[3],arr[4],arr[5]);  
}
```

17.

```
int main()  
{  
}  
  
int arr[6]={11,22,33,44,55,66};  
printf(“%d\n”,++*arr);  
printf(“%d\n”,*arr++);  
  
int main()  
{  
char arr[10]={‘0’,‘1’, ‘2’, ‘3’, ‘4’, ‘5’, ‘6’,‘7’, ‘8’, ‘9’};  
char *cp;  
int *ip;  
cp=ip=arr;  
printf(“%x %x\n”, *cp,*ip);  
cp++;  
ip++;  
printf(“%x %x\n”,*cp,*ip);  
}
```

18.

```
void print(char *ptr, int n)  
{  
if(n)  
print(ptr+1, n-1);  
printf(“%c”,*ptr);  
}
```

```

int main()
{
char arr[]={‘a’, ‘b’, ‘c’, ‘d’, ‘e’};
print(arr,sizeof(arr));
}

```

```

19. main()
{
    int a[ ]={2,3,4,5,6};
    int i=0;
    printf("%d",a[i++]+i[a+1]);
}

```

```

20. main()
{
    int a[3],i;
    for(i=0;i<3;i++)
        a[i]=i++;
    for(i=0;i<3;i++)
        printf("%d",a[i]);
}

```

```

21. int main( )
{
int a[ 5]={11,22,33,44,55};
int b[5];
b=a;
printf(“%d %d\n”,*a,*b);
}

```

```

22. void fun(int *ptr, int n)
{

```

```
if(n){  
    fun(ptr+1,n-1)  
    ++*ptr;  
}  
}  
int main()  
{  
    int arr[ ]={11,22,33,44,55},i;  
    fun(arr,sizeof(arr)/sizeof(*arr));  
    for(i=0;i<5;i++)  
        printf("%d",arr[i]);  
}
```